多项式模板

Lin1043

Version: 1.0

Last update: February 2, 2019

Abstract

最近学习了一发多项式姿势，记一下模板。

1 Introduction

以下多项式模板基于系数mod998244353的多项式，使用**NTT**解决。

代码中出现的KCZ代表模数，g代表模数的原根，$mp(x)$表示$\frac{1}{x} \bmod KCZ$

2 Code

完整代码

$$G(x) \equiv \left(\left(1 + \ln\left(2 + F(x) - F(0) - \exp\left(\int \frac{1}{\sqrt{F(x)}} dx\right)\right)\right)^k\right)' \pmod{x^n}$$

orz挑战多项式

有空就回来写做法（然后就咕了？咕咕咕预定

2.1 NTT

给出一个多项式$F(x)$，在系数和点值两种表示方式之间快速转换。复杂度$O(n\log n)$

```
void NTT(LL *A , int lim , int type)
{
    rep(i , 0 , lim - 1)
    if(i < rev[i]) std::swap(A[i] , A[rev[i]]);
    for(int mid = 1 ; mid < lim ; mid <<= 1)
    {
        LL Wn = mp(g , (KCZ - 1) / (mid << 1));
        for(int R = mid << 1 , j = 0 ; j < lim ; j += R)
        {
            LL *x = A + j , *y = x + mid , w = 1;
            for(int k = 0 ; k < mid ; ++k)
            {
                LL u = *x , v = w * *y % KCZ;
                *x = add(u , v);
                *y = sub(u , v);
                ++x , ++y , w = w * Wn % KCZ;
            }
        }
    }
    if(type == -1)
    {
        LL ginv = mp(lim);
        rep(i , 0 , lim - 1)
        A[i] = A[i] * ginv % KCZ;
        std::reverse(A + 1 , A + lim);
    }
}
```

2.2 多项式求逆

给出一个多项式$F(x)$，在$O(n\log n)$的复杂度求出满足以下条件的$G(x)$

$$F(x)\times G(x)\equiv 1 \bmod x^n$$

```
void PolyInv(const LL *A , LL *B , int n)
```

```
{
    static LL C[N] , D[N];
    if(n == 1)
    {
        B[0] = mp(A[0]);
        return ;
    }
    PolyInv(A , B , (n + 1) >> 1);
    MakeRev(n << 1);
    copy(C , A , n) , copy(D , B , n);
    NTT(C , lim , 1) , NTT(D , lim , 1);
    rep(i , 0 , lim - 1)
    D[i] = sub(2ll , C[i] * D[i] % KCZ) * D[i] % KCZ;
    NTT(D , lim ,-1);
    copy(B , D , n);
}
```

2.3 多项式求导和积分

给出一个多项式$F(x)$，求出$F'(x)$或是$\int F(x)dx$。时间复杂度$O(n)$

```
void deriv(const LL *A , LL *B , int n)
{
    rep(i , 1 , n - 1)
    B[i - 1] = A[i] * i % KCZ;
    B[n - 1] = 0;
}

void inter(const LL *A , LL *B , int n)
{
    rep(i , 1 , n - 1)
    B[i] = A[i - 1] * mp(i) % KCZ;
    B[0] = 0;
}
```

2.4 多项式对数

给出一个多项式$F(x)$，求满足条件的$G(x)$，时间复杂度$O(n\log n)$

$$G(x) \equiv \ln F(x) \bmod x^n$$

```
void PolyLn(const LL *A, LL *B, int n)
{
    static LL C[N], D[N];
    deriv(A, C, n), PolyInv(A, D, n);
    MakeRev(n << 1);
    set(B, n), set(C, n), set(D, n);
    NTT(C, lim, 1), NTT(D, lim, 1);
    rep(i, 0, lim - 1)
    C[i] = C[i] * D[i] % KCZ;
    NTT(C, lim, -1), inter(C, B, n);
}
```

2.5 多项式开根

给出一个多项式$F(x)$，求满足条件的$G(x)$，时间复杂度$O(n\log n)$

$$G^2(x) \equiv F(x) \bmod x^n$$

```
void PolySqrt(const LL *A, LL *B, int n)
{
    static LL C[N], D[N], inv2 = mp(2);
    if(n == 1)
    {
        B[0] = 1;
        return;
    }
    PolySqrt(A, B, (n + 1) >> 1);
    PolyInv(B, D, n), copy(C, A, n);
    MakeRev(n << 1);
    NTT(C, lim, 1), NTT(B, lim, 1), NTT(D, lim, 1);
    rep(i, 0, lim - 1)
    C[i] = (C[i] + B[i] * B[i] % KCZ) % KCZ * inv2 % KCZ * D[i] % KCZ;
    NTT(C, lim, -1), copy(B, C, n);
}
```

2.6 多项式指数

给出一个多项式$F(x)$，求满足条件的$G(x)$，时间复杂度$O(n\log n)$

$$G(x) \equiv e^{F(x)} \bmod x^n$$

记得只copy后一半

```
void PolyExp(const LL *A, LL *B, int n)
{
    static LL C[N], D[N], lnD[N];
    if(n == 1)
    {
        B[0] = 1;
        return;
    }
    PolyExp(A, B, (n + 1) >> 1);
    PolyLn(B, lnD, n), MakeRev(n);
    copy(C, A, n), copy(D, B, n);
    rep(i, 0, n - 1)
    C[i] = sub(C[i], lnD[i]);
    set(C, n), C[0] += 1;
    NTT(C, lim, 1), NTT(D, lim, 1);
    rep(i, 0, lim - 1)
    C[i] = C[i] * D[i] % KCZ;
    NTT(C, lim, -1);
    rep(i, (n + 1) >> 1, n) B[i] = C[i];
}
```

2.7 多项式幂次

给出一个多项式$F(x)$，求满足条件的$G(x)$，时间复杂度$O(n\log n)$

$$G(x) \equiv F^k(x) \bmod x^n$$

```
void PolyPow(const LL *A, LL *B, int n, LL d)
{
    static LL C[N];
    PolyLn(A, C, n);
    rep(i, 0, n - 1)
```

```
    C[i] = C[i] * d % KCZ;
    PolyExp(C , B , n);
}
```

2.8 多项式幂次

给出两个多项式$F(x)$和$G(x)$，求满足条件的$Q(x), R(x)$，时间复杂度$O(n \log n)$

$$F(x) \equiv Q(x) \times G(x) + R(x) \bmod x^n$$

记得把$F(x)$大于d次幂的清零

```
void PolyDiv(const LL *A , const LL *B , LL *C , LL *D , int n , int m)
{
    static LL F[N] , G[N] , GR[N];
    rep(i , 0 , n - 1) F[i] = A[n - 1 - i];
    rep(i , 0 , m - 1) G[i] = B[m - 1 - i];
    int d = n - m + 1;
    PolyInv(G , GR , d);
    MakeRev(n << 1);
    set(F , n) , set(G , m) , set(GR , d);
    NTT(F , lim , 1) , NTT(GR , lim , 1);
    rep(i , 0 , lim - 1)
    F[i] = F[i] * GR[i] % KCZ;
    NTT(F , lim ,-1);
    std::reverse(F , F + d);
    set(F , d) , copy(C , F , d);
    copy(G , B , m);
    NTT(F , lim , 1) , NTT(G , lim , 1);
    rep(i , 0 , lim - 1)
    F[i] = F[i] * G[i] % KCZ;
    NTT(F , lim ,-1);
    rep(i , 0 , m - 1) D[i] = sub(A[i] , F[i]);
}
```